



Python Programming

Ch-4 Classes And Objects

Course Outcome : Design Classes for given problem.

Prof. Perna S. Patil
Department of Computer Engineering
School of Polytechnic & Skill Development

Points to be Covered

Classes And Objects

4.1 Object-Oriented Programming OOP in Python: Classes, ‘self-variable, Overriding Methods, Data hiding, Data Abstraction , Inheritance- Multiple, Hierarchical and Multi-level inheritance

4.2 Error and Exceptions: Difference between an error and Exception, Handling Exception, ‘try except for block, Raising Exceptions, User Defined Exceptions. try except for block, Raising Exceptions, User Defined Exceptions.

Classes And Objects



SINCE 1983

Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Python programming offers OOP style programming and provides an easy way to develop programs.

Python programming uses the OOPs concepts that makes Python more powerful to help design a program that represents real-world entities.

Python also supports OOP concepts such as Inheritance, Method overriding, Data abstraction and Data hiding

Classes And Objects



Important terms in OOP/Terminology of OOP:

- 1. Class:** Classes are defined by the user. The class provides the basic structure for an object. It consists of data members and method members that are used by the instances, (objects) of the class.
- 2. Object:** A unique instance of a data structure that is defined by its class. An object comprises both data members (class variables and instance variables) and methods. Class itself does nothing but the real functionality is achieved through their objects. Object is an instance or occurrence of the class. It takes the properties (variables) and uses the behavior (methods) defined in the class.
- 3. Data Member:** A variable defined in either a class or an object; it holds the data associated with the class or object.
- 4. Instance Variable:** A variable that is defined in a method; its scope is only within the object that defines it.

Classes And Objects



SINCE 1983

Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Function Overloading: A function defined more than one time with different behavior's is known as function overloading. The operation performed varies by the types of objects or arguments involved.

Encapsulation: Encapsulation is the process of binding together the methods and data variables as a single entity i.e., class. This keeps both the data and functionality code safe from the outside world. It hides the data within the class and makes it available only through the methods.

Inheritance: The transfer of the characteristics of a class to other classes that are derived from it. A class 'A' that can use the characteristics of another class 'B' is said to be derived class i.e. a class inherited from B. This process is called inheritance.

Polymorphism: Polymorphism allows one interface to be used for a set of actions i.e., one name may refer to different functionality. The word polymorphism means having many forms. In programming, polymorphism means same function name (but different signatures) being used for different types.

Data Abstraction: The basic idea of data abstraction is to visible only the necessary information, unnecessary information will be hidden from the outside world. Abstraction is a process of hiding the implementation details and showing only functionality to the user. Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where we type the text and send the message. We don't know the internal processing about the message delivery.

Classes And Objects



DATA ENCAPSULATION AND DATA ABSTRACTION

- We can restrict access of methods and variables in a class with the help of encapsulation. It will prevent the data from being modified by accident.
- Encapsulation is used to hide the values or state of a structured data object inside a class, preventing unauthorized parties' direct access to them.
- Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.
- The terms encapsulation and abstraction (data hiding) are often used as synonyms. Data abstraction we can achieve through encapsulation.
Encapsulation is a process to bind data and functions together into a single unit i.e., class while as abstraction is a process in which the data inside the class is hidden from the outside world. It hides the sensitive information.
- Hiding internal details and showing functionality is known as data abstraction.
- To support encapsulation, declare the methods or variables as private in the class. The private methods cannot be called by the object directly. It can be called only from within the class in which they are defined.
- Any function prefix with double underscore is called private method which is accessible only within the class where it is declared.

Classes And Objects



Dr. Vishwanath Karad

MIT WORLD PEACE
UNIVERSITY | PUNE
SOCIAL INNOVATION & PARTNERSHIPS

SINCE 1983

Supported by
UNESCO
UNESCO Chair for
Human Rights, Democracy, Peace & Tolerance
World Peace Centre (Alandi) Pune, India

Sr. No.	Types	Description
1.	Public methods	Accessible from anywhere i.e. inside the class in which they are defined, in the sub class, in the same script file as well as outside the script
2	Private methods	Accessible only in their own class. starts with two underscores.
3	Public variables	Accessible from anywhere.
4	Private variables	Accessible only in their own class or by a method if defined. starts with two underscores.

Classes And Objects



CLASSES

- Python is an object oriented programming language. Almost everything in Python is an object, with its properties and methods.
- Object is simply a collection of data (variables) and methods (functions) that act on those data.
- A class is like an object constructor or a "blueprint" for creating objects. A class defines the properties and behavior (variables and methods) that is shared by all its objects.

Create a Class

To create a class, use the keyword class:

Example

Create a class named MyClass, with a property named x:

```
class MyClass:
```

```
    x = 5
```

Create Object

Now we can use the class named MyClass to create objects:

Example

Create an object named p1, and print the value of x:

```
p1 = MyClass()
```

```
print(p1.x)
```

Classes And Objects

The `__init__()` Function

The examples above are classes and objects in their simplest form, and are not really useful in real life applications.

To understand the meaning of classes we have to understand the built-in `__init__()` function.

All classes have a function called `__init__()`, which is always executed when the class is being initiated.

Use the `__init__()` function to assign values to object properties, or other operations that are necessary to do when the object is being created:

Note: The `__init__()` function is called automatically every time the class is being used to create a new object. By using the “**self**” keyword we can access the attributes and methods of the class in **python**. It binds the attributes with the given arguments. **self** is parameter in function and user can use another parameter name in place of it. But it is advisable to use **self** because it increase the readability of code.

Example

Create a class named Person, use the `__init__()` function to assign values for name and age:

```
class Person:
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
p1 = Person("John", 36)
```

```
print(p1.name)  
print(p1.age)
```

Classes And Objects

Constructors:

In Python, the method the `__init__()` simulates the constructor of the class. This method is called when the class is instantiated.

It accepts the **self**-keyword as a first argument which allows accessing the attributes or method of the class.

We can pass any number of arguments at the time of creating the class object, depending upon the `__init__()` definition. It is mostly used to initialize the class attributes.

Every class must have a constructor, even if it simply relies on the default constructor.

Syntax:

```
def __init__(self):
```

```
# body of the constructor
```

· `__init__` is a special method in Python classes, which is the constructor method for a class.

```
class Employee:
```

```
    def __init__(self, name, id):
```

```
        self.id = id
```

```
        self.name = name
```

```
    def display(self):
```

```
        print("ID: %d \nName: %s" % (self.id, self.name))
```

```
emp1 = Employee("John", 101)
```

```
emp2 = Employee("David", 102)
```

```
# accessing display() method to print employee 1 information
```

```
emp1.display()
```

```
# accessing display() method to print employee 2 information
```

```
emp2.display()
```

Classes And Objects

The **types of constructors** includes default constructor and parameterized constructor.

1. Default constructor:

- The default constructor is simple constructor which does not accept any arguments. It's definition has only one argument which is a reference to the instance being constructed.

Display Hello message using default constructor.

```
class Student:
def __init__(self):
print("This is non parametrized constructor")
def show(self,name):
print("Hello",name)
s1 = Student()
s1.show("Meenakshi")
```

For parameterized constructor.

```
class Student:
def __init__(self,name):
print("This is parametrized constructor")
self.name = name
def show(self):
print("Hello",self.name)
s1 = Student("Meenakshi")
s1.show()
```

The parameterized constructor take its first argument as a reference to the instance being constructed known as self and the rest of the arguments are provided by the programmer

Classes And Objects

Destructor:

- A class can define a special method called a destructor with the help of `__del__()`. It is invoked automatically when the instance (object) is about to be destroyed.
- It is mostly used to clean up any non-memory resources used by an instance (object).

Example: For destructor.

```
class Student:
def __init__(self):
print('non parameterized constructor-student created ')
def __del__(self):
print('Destructor called, student deleted.')
s1=Student()
s2=Student()
del s1
```

Classes And Objects

Method Overriding

- Overriding is the ability of a class to change the implementation of a method provided by one of its base class. Method overriding is thus a strict part of the inheritance mechanism.
- To override a method in the base class, we must define a new method with same name and same parameters in the derived class.
- Overriding is a very important part of OOP since it is the feature that makes inheritance exploit its full power. Through method overriding a class may "copy" another class, avoiding duplicated code, and at the same time enhance or customize part of it.

Example: For method overloading.

```
class A: # parent class
```

```
"Parent Class"
```

```
def display(self):
```

```
print ('This is base class.')
```

```
class B(A): #derived class
```

```
"Child/Derived class"
```

```
def display(self):
```

```
print ('This is derived class.')
```

```
obj = B() # instance of child
```

```
obj.display() # child calls overridden method
```

Output:

This is derived class.



Classes And Objects

Inheritance In inheritance objects of one class procure the properties of objects of another class.

Inheritance provide code usability, which means that some of the new features can be added to the code while using the existing code.

The mechanism of designing or constructing classes from other classes is called inheritance.

- The new class is called derived class or child class and the class from which this derived class has been inherited is the base class or parent class.
- In inheritance, the child class acquires the properties and can access all the data members and functions defined in the parent class. A child class can also provide its specific implementation to the functions of the parent class.

Syntax:

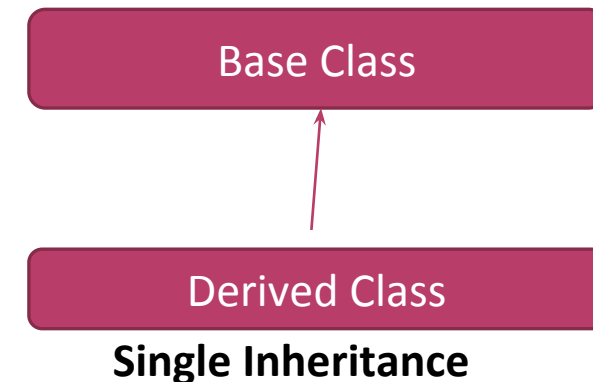
```
class A:
```

```
# properties of class A
```

```
class B(A):
```

```
# class B inheriting property of class A
```

```
# more properties of class B
```



Classes And Objects



SINCE 1983

Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



```
class Vehicle:
    name="Maruti"
    def display(self):
        print("Name= ",self.name)
    class Category(Vehicle):
        price=2000
        def disp_price(self):
            print("Price=$",self.price)
    car1=Category()
    car1.display()
    car1.disp_price()
```

Classes And Objects

Multilevel Inheritance:

- Multi-level inheritance is archived when a derived class inherits another derived class.

There is no limit on the number of levels up to which, the multi-level inheritance is archived in python.

- In multilevel inheritance we inherit the classes at multiple separate levels.

We have three classes A, B and C, where A is the super class, B is its sub(child) class and C is the sub class of B.

Syntax:

class A:

properties of class A

Fig. 5.2

class B(A):

class B inheriting property of class A

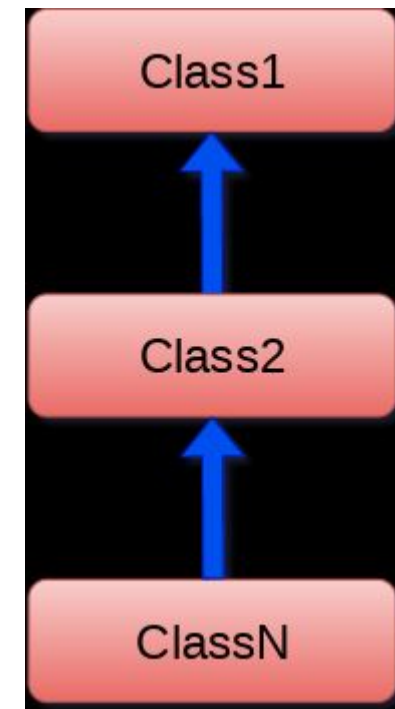
more properties of class B

class C(B):

class C inheriting property of class B

thus, class C also inherits properties of class A

more properties of class C



Classes And Objects

Example: For multilevel inheritance.

```
class Grandfather: (A)
```

```
def display1(self):
```

```
print("Grand Father")
```

```
class Father(Grandfather): class B(A)
```

```
def display2(self):
```

```
print("Father")
```

```
#The child class Son inherits another child class Father
```

```
class Son(Father): Class C(B)
```

```
def display3(self):
```

```
print("Son")
```

```
s1=Son()
```

```
s1.display3()
```

```
s1.display2()
```

```
s1.display1()
```

Classes And Objects

Multiple Inheritance:

- Python provides us the flexibility to inherit multiple base classes in the child class.
- Multiple Inheritance means that we are inheriting the property of multiple classes into one. In case we have two classes, say A and B, and we want to create a new class which inherits the properties of both A and B.
- So it just like a child inherits characteristics from both mother and father, in python, we can inherit multiple classes in a single child class.

Syntax:

class A:

variable of class A

functions of class A

class B:

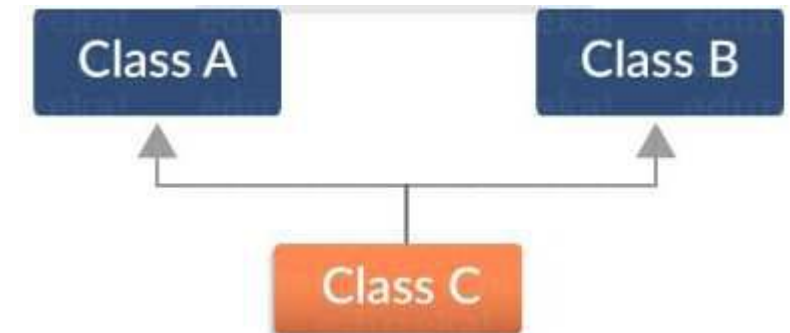
variable of class A

functions of class A

class C(A, B):

class C inheriting property of both classA and B

add more properties to class C



Classes And Objects

Example:

```
# base class
class Father:
def display1(self):
print("Father")
#base class
class Mother:
def display2(self):
print("Mother")
#derived class
class Son(Father,Mother):
def display3(self):
print("Son")
s1=Son()
s1.display3()
s1.display2()
s1.display1()
```

Classes And Objects

Hierarchical Inheritance:

- When more than one derived classes are created from a single base – it is called hierarchical inheritance.

In following program, we have a parent (base) class name Email and two child (derived) classes named Gmail and yahoo.

Example: For hierarchical inheritance.

class Email:

```
def send_email(self, msg):  
    print()
```

```
class Gmail(Email):
```

```
def send_email(self, msg):  
    print( "Sending `{}` from Gmail".format(msg) )
```

```
class Yahoo(Email):
```

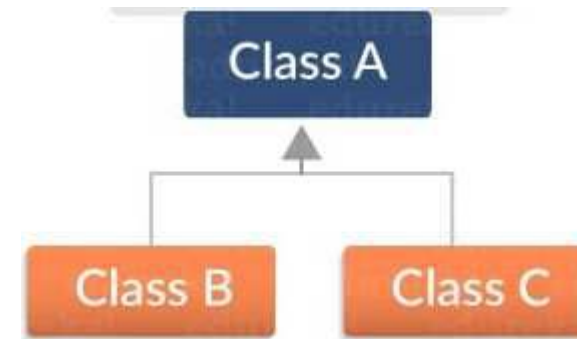
```
def send_email(self, msg):  
    print( "Sending `{}` from Yahoo".format(msg) )
```

```
client1 = Gmail()
```

```
client1.send_email("Hello!")
```

```
client2 = Yahoo()
```

```
client2.send_email("Hello!")
```



Classes And Objects

EXCEPTION HANDLING

When we execute a Python program, there may be a few uncertain conditions which occur, known as errors. Errors also referred to as bugs that are incorrect or inaccurate action that may cause the problems in the running of the program or may interrupt the execution of the program. The programmer may provide wrong result of the execution of the program.

There are three types of error that occur:

- 1. Compile Time error:** Occurs at the time of compilation, include due to the violation of syntax rules like missing of a colon (:) error occur.
- 2. Runtime Errors:** Occurs during the runtime of a program, explain include error occur due to wrong input submitted to program by user.
- 3. Logical Errors:** Occurs due to wrong logic written in the program.
 - Errors occur at runtime are known as exception. Errors detected during execution of program.
 - Python provides a feature (Exception handling) for handling any unreported errors in program.
 -

Classes And Objects



When exception occurs in the program, execution gets terminated. In such cases we get system generated error message.

- By handling the exceptions, we can provide a meaningful message to the user about the problem rather than system generated error message, which may not be understandable to the user.
- **Exception can be either built-in exceptions or user defined exceptions.**
- The interpreter or built-in functions can generate the built-in exceptions while user defined exceptions are custom exceptions created by the user.

An exception is also called ad runtime error that can halt the execution of the program.

- An exception is an error that happens/occurs during execution of a program. When that error occurs, Python generate an exception that can be handled, which avoids the normal flow of the program's instructions.
- Errors detected during execution are called exceptions. An exception is an event (usually an error), which occurs during the execution of a program that disrupts the normal flow of execution as the program.
- In Python programming we can handle exceptions using try-except statement, try-finally statement and raise statement.

Classes And Objects

Following table list all the standard exceptions available in Python programming language:

Sr. No.	Exception	Cause of Error
1.	ArithmeticError	Base class for all errors that occur for numeric calculation.
2.	AssertionError	Raised in case of failure of the assert statement.
3.	AttributeError	Raised in case of failure of attribute reference or assignment.
4.	Exception	Base class for all exceptions.
5.	EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
6.	EnvironmentError	Base class for all exceptions that occur outside the Python environment.
7.	FloatingPointError	Raised when a floating point calculation fails.
8.	ImportError	Raised when an import statement fails.
9.	IndexError	Raised when an index is not found in a sequence.
10.	IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
11.	IndentationError	Raised when indentation is not specified properly.
12.	KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
13.	KeyError	Raised when the specified key is not found in the dictionary.
14.	LookupError	Base class for all lookup errors.
15.	NameError	Raised when an identifier is not found in the local or global namespace.
16.	NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.



Dr. Vishwanath Karad

SINCE 1983

D PEACE
TY | PUNE
INNOVATION & PARTNERSHIPS

Supported by
UNESCO
UNESCO Chair for
Human Rights, Democracy, Peace & Tolerance
World Peace Centre (Alandi) Pune, India

Classes And Objects

17.	<code>OverflowError</code>	Raised when a calculation exceeds maximum limit for a numeric type.
18.	<code>OSError</code>	Raised for operating system-related errors.
19.	<code>RuntimeError</code>	Raised when a generated error does not fall into any category.
20.	<code>StopIteration</code>	Raised when the <code>next()</code> method of an iterator does not point to any object.
21.	<code>SystemExit</code>	Raised by the <code>sys.exit()</code> function.
22.	<code>StandardError</code>	Base class for all built-in exceptions except <code>StopIteration</code> and <code>SystemExit</code> .
23.	<code>SyntaxError</code>	Raised when there is an error in Python syntax.
24.	<code>SystemError</code>	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
25.	<code>SystemExit</code>	Raised when Python interpreter is quit by using the <code>sys.exit()</code> function. If not handled in the code, causes the interpreter to exit.
26.	<code>TypeError</code>	Raised when an operation or function is attempted that is invalid for the specified data type.
27.	<code>UnboundLocalError</code>	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
28.	<code>ValueError</code>	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
29.	<code>ZeroDivisionError</code>	Raised when division or modulo by zero takes place for all numeric types.

Classes And Objects



SINCE 1983

Dr. Vishwanath Karad
MIT WORLD PEACE
UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Exception Handling in Python Programming

- An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions.
- In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.
- For example, if function A calls function B which in turn calls function C and an exception occurs in function C. If it is not handled in C, the exception passes to B and then to A.

When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

- The exception handling is a process that provides a way to handle exceptions that occur at runtime.
- The exception handling is done by writing exception handlers in the program.

The exception handlers are blocks that execute when some exception occurs at runtime.

Exception handlers displays same message that represents information about the exception.

Classes And Objects



Try-except

- In Python, exceptions can be handled using a try statement. A try block consisting of one or more statements is used by programmers to partition code that might be affected by an exception.
- A critical operation which can raise exception is placed inside the try clause and the code that handles exception is written in except clause.

For handling exception in Python, the exception handler block needs to be written which consists of set of statements that need to be executed according to raised exception. There are three blocks that are used in the exception handling process, namely, try, except and finally.

1. try Block: A set of statements that may cause error during runtime are to be written in the try block.

2. except Block: It is written to display the execution details to the user when certain exception occurs in the program. The except block executed only when a certain type as exception occurs in the execution of statements written in the try block.

3. finally Block: This is the last block written while writing an exception handler in the program which indicates the set of statements that many use to clean up to resources used by the program.

Classes And Objects



SINCE 1983

Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Syntax:

try:

D the operations here

.....

except Exception1:

If there is Exception1, then execute this block.

except Exception2:

If there is Exception2, then execute this block.

.....

else:

If there is no exception then execute this block.

Example: For try-except clause/statement.

try:

```
fh = open("testfile", "w")
```

```
fh.write("This is my test file for exception  
handling!!")
```

```
except IOError:
```

```
print ("Error: can\'t find file or read data")
```

```
else:
```

```
print ("Written content in the file successfully")
```

```
fh.close()
```

Classes And Objects



SINCE 1983

Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Example: For try statement.

```
n=10
```

```
m=0
```

```
try:
```

```
n/m
```

```
except ZeroDivisionError:
```

```
print("Divide by zero error")
```

```
else:
```

```
print (n/m)
```

Output:

Divide by zero error

Classes And Objects

try-Except with No Exception

- We can use try-except clause with no exception. All types of exceptions that occur are caught by the try-except statement.
- However, because it catches all exceptions, the programmer cannot identify the root cause of a problem that may occur. Hence this type of programming approach is not considered good.

Syntax:

try:

 D the operations here

.....

except:

 If there is any exception, then execute this block.

.....

else:

 If there is no exception then execute this block.

Example: For try-except statement with no exception.
while True:

 try:

 a=int(input("Enter an integer: "))

 div=10/a

 break

 except:

 print("Error Occurred")

 print("Please enter valid value")

 print()

print("Division is: ",div)

Classes And Objects

try...finally

The try statement in Python can have an optional finally clause. This clause is executed no matter what, and is generally used to release external resources.

- The statement written in finally clause will always be executed by the interpreter, whether the try statement raises an exception or not.
- A finally block is always executed before leaving the try statement, whether an exception is occurred or not. When an exception is occurred in try block and has not been handled by an except block, it is re-raised after the finally block has been executed.
- The finally clause is also executed “on the way out” when any other clause of the try statement is left via a break, continue or return statement.

Syntax:

try:

 D the operations here;

.....

Due to any exception, this may be skipped.

finally:

 This would always be executed.

Classes And Objects



SINCE 1983

Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Example: For try-finally.

try:

```
fh = open("testfile", "w")
```

```
fh.write("This is my test file for exception handling!!")
```

finally:

```
print ("file is closing")
```

```
fh.close()
```

Classes And Objects



SINCE 1983

Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



raise Statement

We can raise an existing exception by using raise keyword. So, we just simply write raise keyword and then the name of the exception.

- The raise statement allows the programmer to force a specified exception to occur.
- For example: We can use raise to throw an exception if age is less than 18 condition occurs.

while True:

 try:

 age = int(input("Enter your age for election: "))

 if age < 18:

 raise Exception

 else:

 print("you are eligible for election")

 break

except Exception:

 print("This value is too small, try again")

Classes And Objects



SINCE 1983

Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



User Defined Exception

- Python has many built-in exceptions which forces the program to output an error when something in it goes wrong. However, sometimes we may need to create custom exceptions that serves the purpose.
- Python allow programmers to create their own exception class. Exceptions should typically be derived from the Exception class, either directly or indirectly. Most of the built-in exceptions are also derived from Exception class.
- User can also create and raise his/her own exception known as user defined exception.
- In the following example, we create custom exception class AgeSmallException that is derived from the base class Exception.

Classes And Objects

Example 1: Raise a user defined exception if age is less than 18.

```
# define Python user-defined exceptions
class Error(Exception):
    """Base class for other exceptions"""
    pass
class AgeSmallException(Error):
    """Raised when the input value is too small"""
    pass
# main program
while True:
    try:
        age = int(input("Enter your age for election: "))
        if age < 18:
            raise AgeSmallException
        else:
            print("you are eligible for election")
```

```
        break
    except AgeSmallException:
        print("This value is too small, try
        again!")
        print()
```

Classes And Objects

Example 2: Raise a user defined exception id password is incorrect.

```
class InvalidPassword(Exception):  
    pass  
def verify_password(pswd):  
    if str(pswd) != "abc":  
        raise InvalidPassword  
else:  
    print('Valid Password: '+str(pswd))  
# main program  
verify_password("abc") # won't raise exception  
verify_password("xyz") # will raise exception
```

Classes And Objects



SINCE 1983

Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

